

Barreras.pdf



JesusLagares



Programación Concurrente y de Tiempo Real



3º Grado en Ingeniería Informática



**Escuela Superior de Ingeniería
Universidad de Cádiz**



[Accede al documento original](#)

antes



**Descarga sin publi
con 1 coin**



Después

WUOLAH



Importante

Puedo eliminar la publi de este documento con 1 coin

¿Cómo consigo coins? → Plan Turbo: barato
→ Planes pro: más coins

pierdo
espacio



Barreras



Una **barrera** es una primitiva de sincronización que establece un **punto de encuentro** para varias hebras o procesos: todas deben llegar a la barrera antes de que cualquiera pueda continuar.

Explicación

Con la barrera podemos sincronizar fases de ejecución. Por ejemplo:

- Todas las hebras ejecutan trabajo de la Fase 1.
- Cuando terminan, llaman a la barrera.
- Nadie puede comenzar la Fase 2 hasta que todas hayan llegado a la barrera.

Al igual que ocurre con otras primitivas tenemos que diferenciar entre el concepto teórico y el de implementación.

El concepto teórico es lo que hemos descrito previamente.

En términos de implementación, podemos encontrarlas tanto en Java (alto nivel) como en C++:

Java

API Standard

En la API standard podemos SIMULAR su comportamiento. No tenemos implementación directa de barreras.

Aquí tenemos un ejemplo de una barrera simulada usando un monitor:

```
public class Barrera {  
  
    private final int N;    // Número de hebras que deben llegar
```

```

private int count = 0; // Cuántas han llegado en esta fase

public Barrera(int N) {
    this.N = N;
}

public synchronized void await() throws InterruptedException {
    count++;

    // Si no soy el último, espero
    if (count < N) {
        while (count < N) {
            wait(); // me bloqueo
        }
    }
    // Si soy el último, despierto a todos
    else {
        count = 0; // reset para que sea cíclica
        notifyAll(); // despierto a todos
    }
}
}

```

API Alto Nivel

En la API de alto nivel contamos con una implementación a través de la clase

`CyclicBarrier`

- Se inicializa con un **número de hebras** (`parties`) que deben llegar.
- Cada hebra llama a `.await()` cuando llega a la barrera.
- Cuando han llegado todas:
 - la barrera se "abre".
 - Este concepto es importante porque la última hebra que llega NO ESPERA, sino que continua directamente y hace que el resto continúe.
 - todas las hebras continúan.

- Es **reutilizable**: se puede usar en múltiples fases (de ahí "cyclic").

Solo esperan las hebras que usan la barrera, es decir, las que llamen a `.await()`.

```
import java.util.concurrent.BrokenBarrierException;
import java.util.concurrent.CyclicBarrier;

public class EjemploCyclicBarrier {

    private static final int NUM_HEBRAS = 3;

    public static void main(String[] args) {

        CyclicBarrier barrera = new CyclicBarrier(NUM_HEBRAS, () -> {
            // Acción opcional cuando todas las hebras alcanzan la barrera
            System.out.println(">> Todas las hebras han llegado a la barrera. Pasa-  
mos a la siguiente fase.");
        });

        for (int i = 0; i < NUM_HEBRAS; i++) {
            int id = i;
            new Thread(() -> trabajo(id, barrera)).start();
        }
    }

    private static void trabajo(int id, CyclicBarrier barrera) {
        try {
            System.out.println("Hebra " + id + " - Fase 1");
            Thread.sleep(500 + id * 200); // simulamos trabajo distinto

            System.out.println("Hebra " + id + " esperando en la barrera...");
            barrera.await(); // punto de sincronización

            // Todos han pasado la barrera
            System.out.println("Hebra " + id + " - Fase 2");
        }
    }
}
```

Importante

Puedo eliminar la publi de este documento con 1 coin

¿Cómo consigo coins? → Plan Turbo: barato
→ Planes pro: más coins

pierdo espacio



Necesito concentración

ali ali oohh
esto con 1 coin me
lo quito yo...

WUOLAH

```
} catch (InterruptedException | BrokenBarrierException e) {  
    e.printStackTrace();  
}  
}  
}
```

En la implementación de alto nivel, las hebras que esperas son aquellas que utilizan la barrera (es decir, las que invocan al método `.await()`).

Sin embargo, también podemos encontrarnos barreras cuando usamos MPJ Express.

MPJ Express

Tenemos la operación **de comunicación colectiva** `Barrier()` sobre un comunicador.

```
MPI.COMM_WORLD.Barrier();
```

Las barreras de MPJ hacen que TODOS los procesos esperen (todos los que correspondan al comunicador que invoca la barrera claro). Y ninguno continúa hasta que TODOS los procesos del comunicador han llegado.

Por tanto, si tan solo uno de los procesos NO LLAMA A LA BARRERA, el sistema quedará bloqueado.

```
import mpi.*;  
  
public class EjemploMPJBarrier {  
    public static void main(String[] args) throws Exception {  
        MPI.Init(args);  
  
        int rank = MPI.COMM_WORLD.Rank();  
        int size = MPI.COMM_WORLD.Size();  
  
        System.out.println("Proceso " + rank + " antes de la barrera");  
  
        // Barrera colectiva: afecta a TODOS los procesos del comunicador  
        MPI.COMM_WORLD.Barrier();  
    }  
}
```

```

        System.out.println("Proceso " + rank + " después de la barrera");

        MPI.Finalize();
    }
}

```

C++

En C++ las barreras pertenecen a la biblioteca estándar desde C++20, dentro de:

```

#include <barrier>
#include <latch>

```

std::barrier (barrera cíclica)

Es la versión C++ de `CyclicBarrier` :

- Se inicializa con el **número de hilos participantes**.
- Cada hilo llama a `arrive_and_wait()` .
- Cuando llegan todos:
 - la barrera se abre,
 - todos continúan,
 - y **se puede reutilizar** para la siguiente fase.

```

#include <iostream>
#include <thread>
#include <vector>
#include <barrier>

const int N = 3;

int main() {
    std::barrier barrera(N, []() {

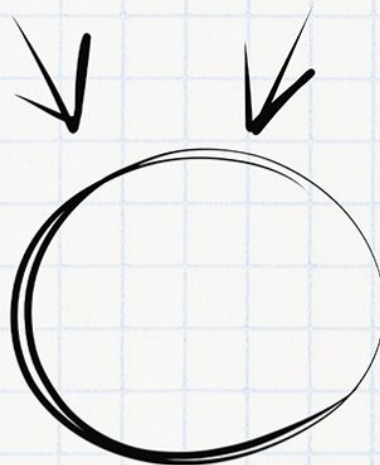
```


Imagínate aprobando el examen

Necesitas tiempo y concentración

Planes	 PLAN TURBO	 PLAN PRO	 PLAN PRO+
 Descargas sin publi al mes	10 	40 	80 
 Elimina el video entre descargas			
 Descarga carpetas			
 Descarga archivos grandes			
 Visualiza apuntes online sin publi			
 Elimina toda la publi web			
 Precios Anual ☐	0,99 € / mes	3,99 € / mes	7,99 € / mes

Ahora que puedes conseguirlo,
¿Qué nota vas a sacar?



WUOLAH

```

        std::cout << ">> Todas las hebras han llegado a la barrera. Nueva fase.\n";
    });

    auto trabajo = [&](int id) {
        std::cout << "Hebra " << id << " - Fase 1\n";
        std::this_thread::sleep_for(std::chrono::milliseconds(500 + id * 200));

        std::cout << "Hebra " << id << " esperando en la barrera...\n";
        barrera.arrive_and_wait(); // Punto de sincronización

        std::cout << "Hebra " << id << " - Fase 2\n";
    };

    std::vector<std::thread> hilos;
    for (int i = 0; i < N; ++i) {
        hilos.emplace_back(trabajo, i);
    }

    for (auto& h : hilos) {
        h.join();
    }
}

```

std::latch (NO es cíclica)

Esto es como `CountDownLatch` de Java:

- Se inicializa con un contador.
- Cada hilo llama a `count_down()`.
- Los que llaman a `wait()` se bloquean hasta que el contador llega a 0.
- **✗ No se puede reutilizar.**

```

#include <iostream>
#include <thread>
#include <vector>

```


Importante

Puedo eliminar la publi de este documento con 1 coin

¿Cómo consigo coins? → Plan Turbo: barato
→ Planes pro: más coins

pierdo
espacio



Necesito
concentración

ali ali ooh
esto con 1 coin me
lo quito yo...

WUOLAH

```
#include <latch>
```

```
int main() {
```

```
    std::latch latch(3);
```

```
    auto trabajo = [&](int id) {
```

```
        std::cout << "Hebra " << id << " trabajando...\n";
```

```
        std::this_thread::sleep_for(std::chrono::milliseconds(500));
```

```
        latch.count_down(); // aviso de que he terminado
```

```
    };
```

```
    std::thread t1(trabajo, 1);
```

```
    std::thread t2(trabajo, 2);
```

```
    std::thread t3(trabajo, 3);
```

```
    latch.wait(); // espera a que TODOS terminen
```

```
    std::cout << ">> Todas las hebras han terminado.\n";
```

```
    t1.join(); t2.join(); t3.join();
```

```
}
```